# WHAT-WHY-HOW

A practical model for understanding software requirements

v0.1.1 (2026-04-27)

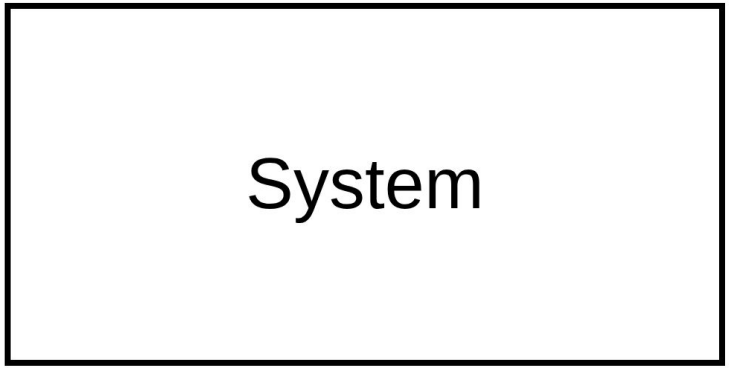
Stanislav Pankevich, StrictDoc Project

# What is this about? Why should I care?

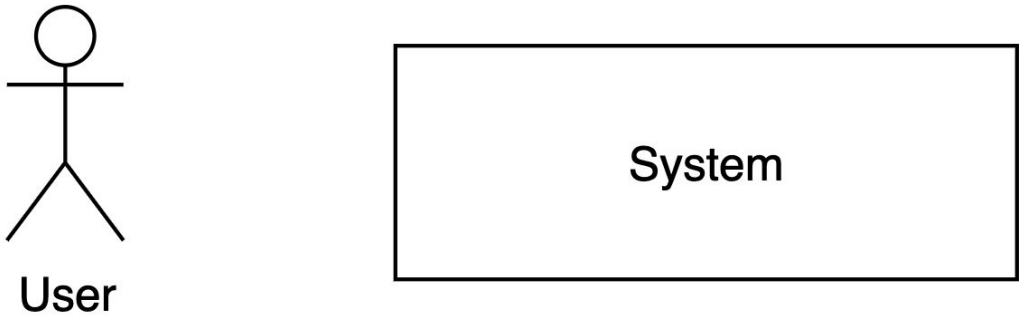
- This presentation is a very quick journey through the topics of:
  1. Software systems engineering
  2. Software requirements
  3. Software architecture
- It could be a good introduction to these topics for those unfamiliar with one or more of them
- For those who are familiar, it could be a simple refresher or a good reinforcement of well-known concepts

# Basic building blocks for systems engineering

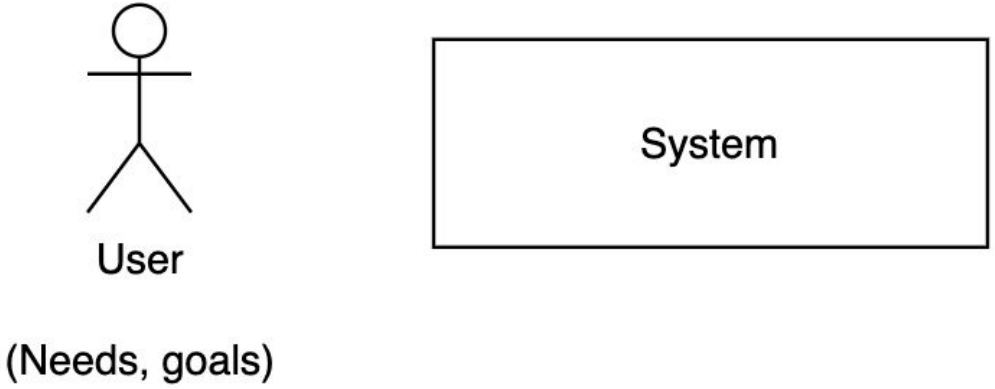
# Let's talk about a system



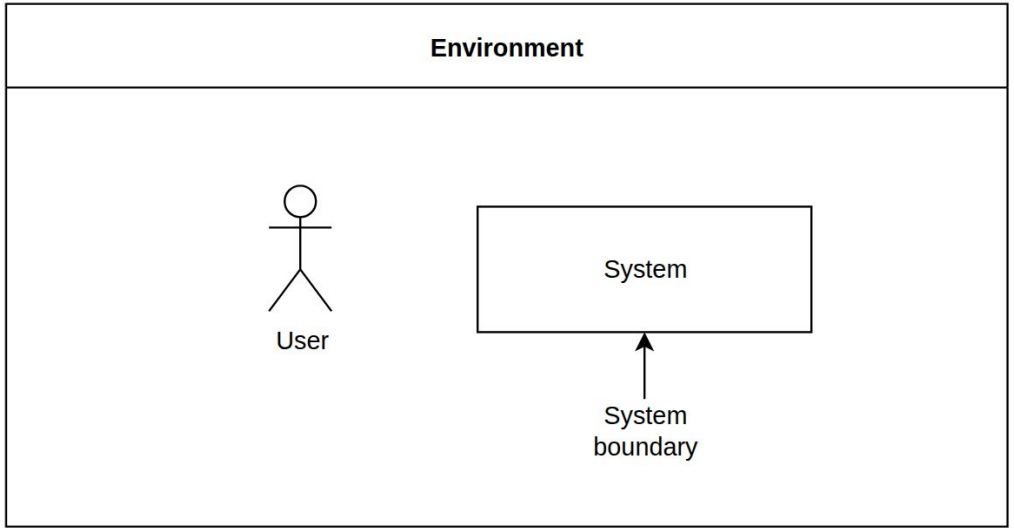
# The system has a user



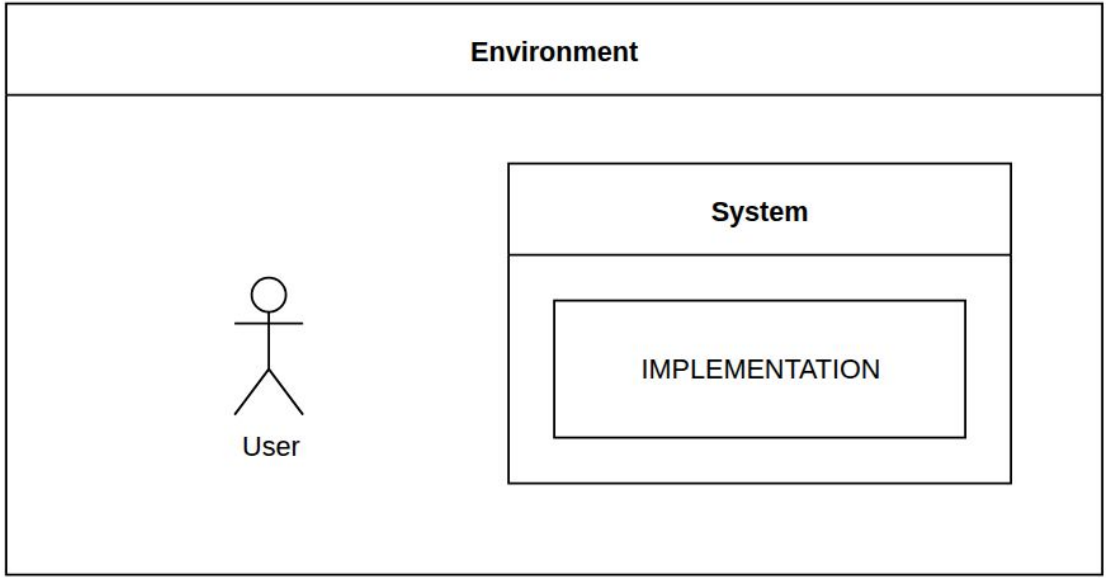
# The user has needs/goals that the system can address



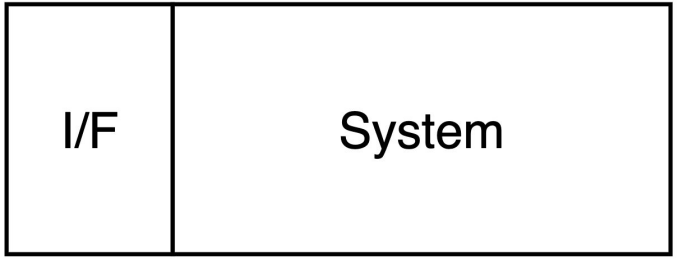
# System environment matters



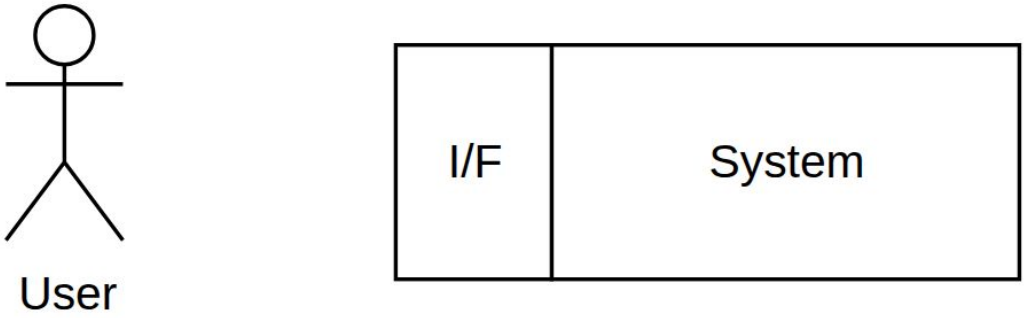
# System implementation



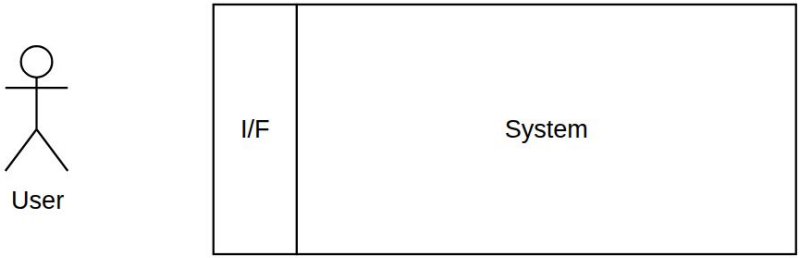
# A system usually has an interface



# A user uses the interface to control/access the system



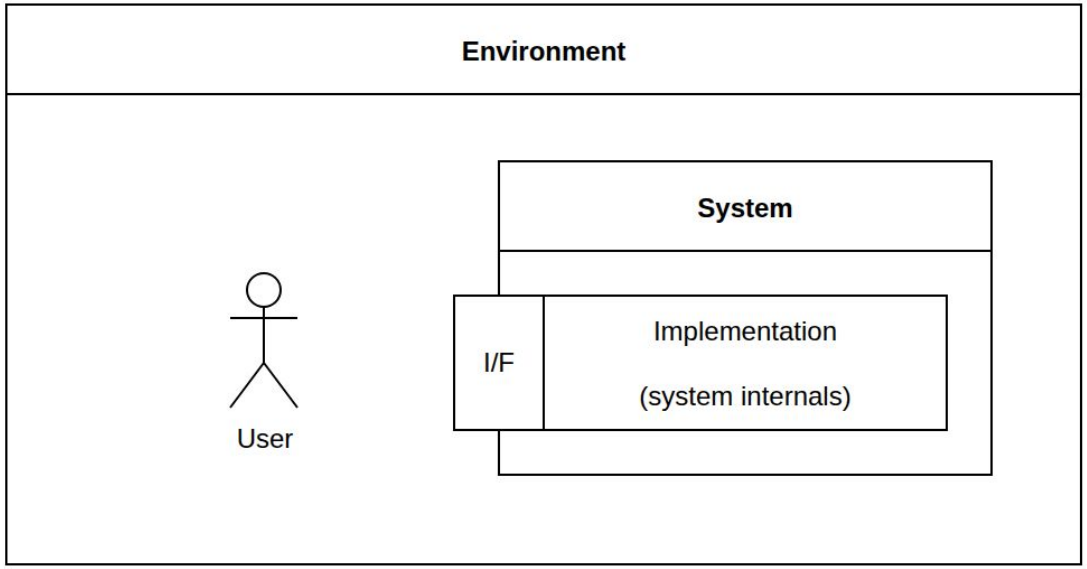
# Interface hides/encapsulates complexity



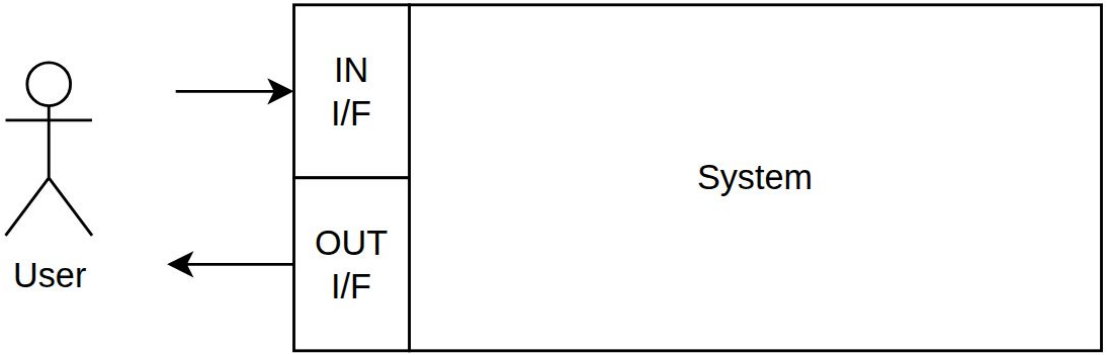

**Small interface, rich functionality:**

A good interface is small and simple, exposing only the minimal surface needed for the user to interact with the system. Behind it, the system implementation can be large and complex, encapsulating the internal logic while delivering substantial value through that small interface.

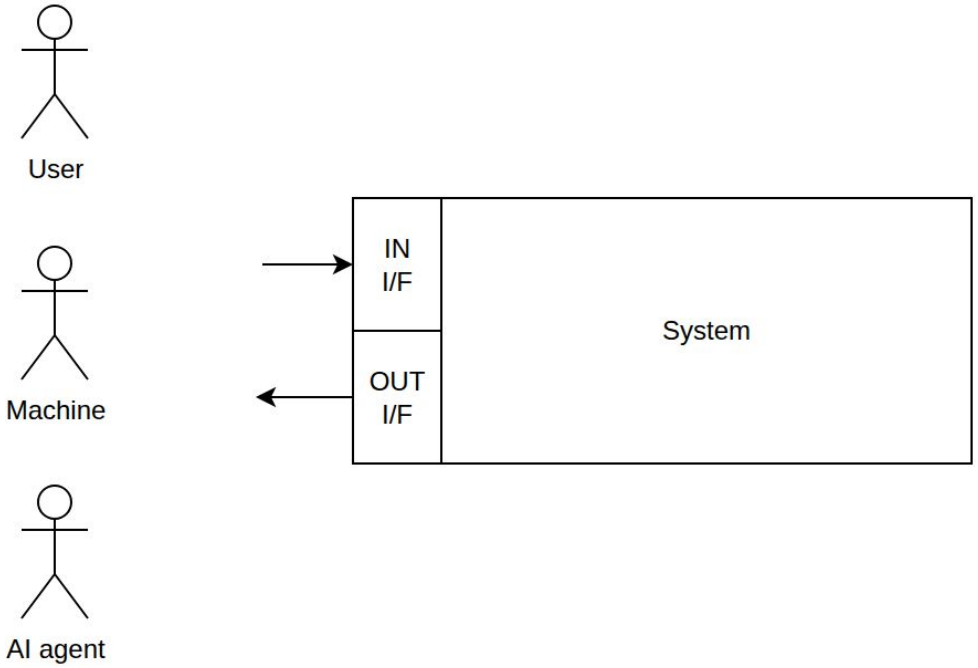
# Interface is what connects a user with the system



# Interface I/O model

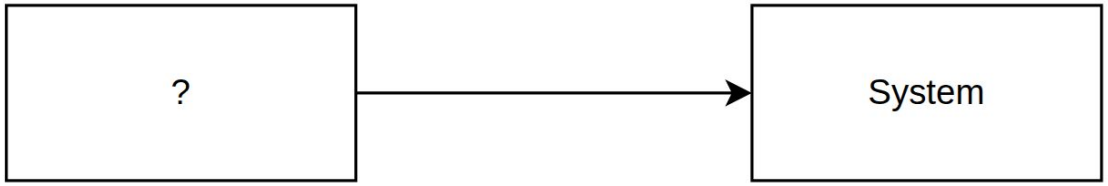


# User can be another system

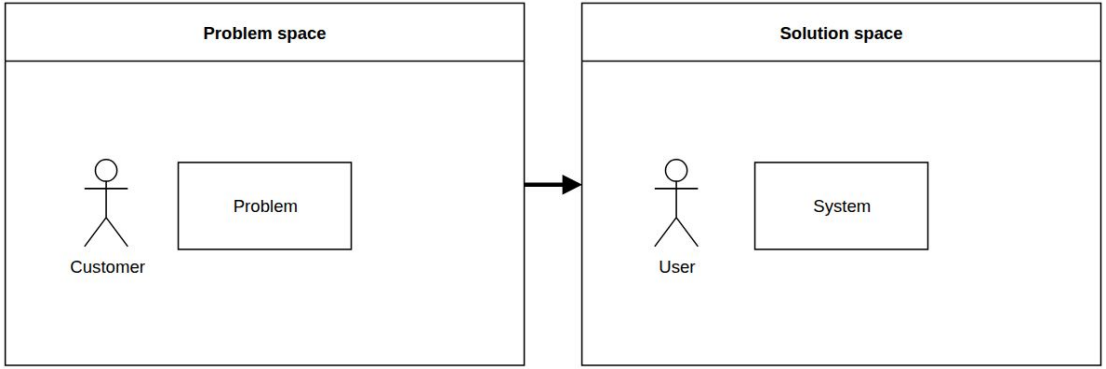


# Problem space vs solution space

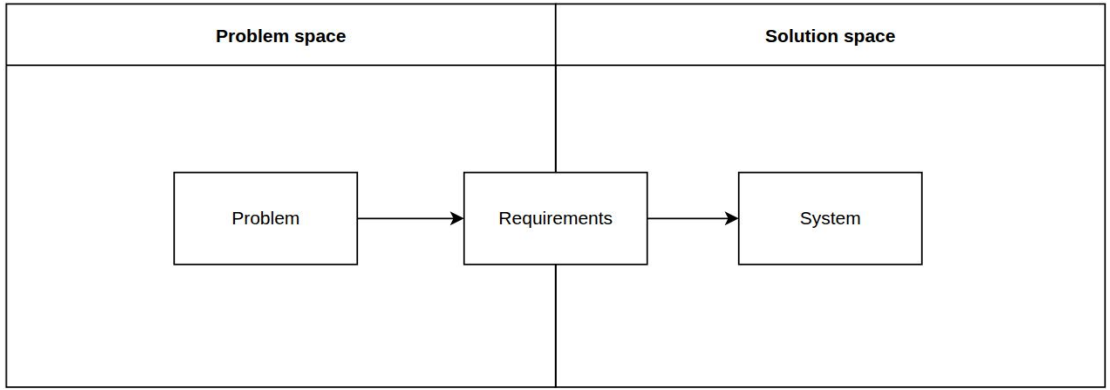
# Where does the system come from?



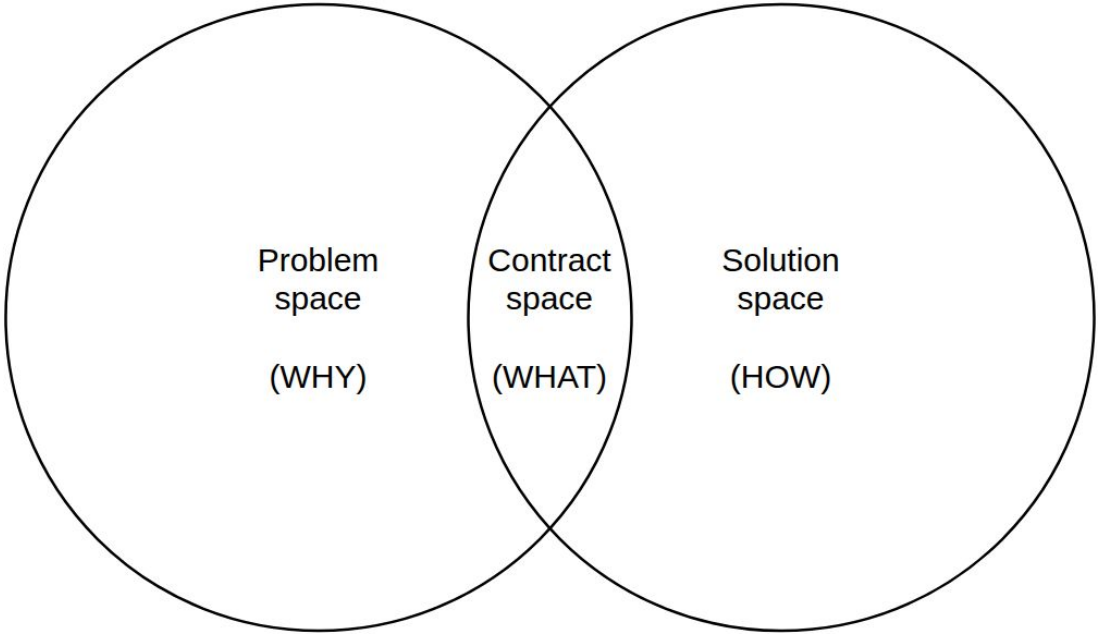
# Problem space vs solution space



# Problem → Requirements → Solution

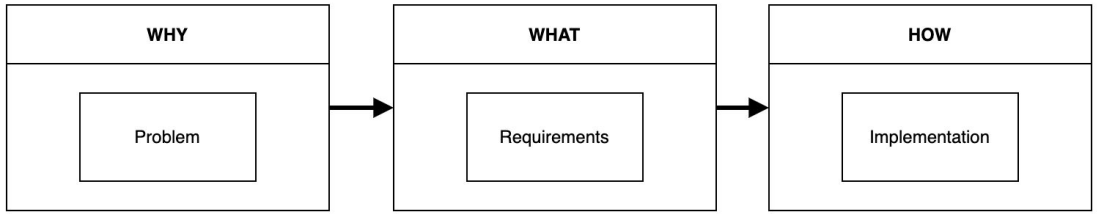


# Problem space vs contract space vs solution space

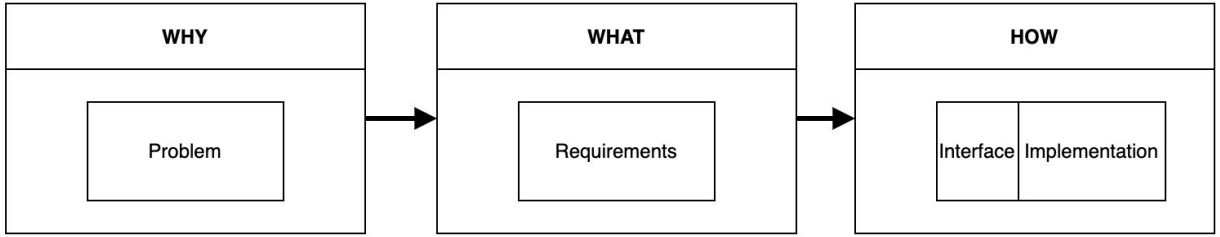


# WHAT-WHY-HOW

# Let's not get lost! It is really about WHAT-WHY-HOW!



# WHY→WHAT→HOW (and INTERFACE)



# WHAT-WHY-HOW — Easy? Or not really?

- People may generally agree that all four aspects of WHAT-WHY-HOW-IF exist, but they do not always agree that they should be distinguished individually, and they mix them together, neglecting the process of separation.

- Intuitively, separating them seems simple, but in practice, it is not an easy task. Even when one attempts to separate them, problems often arise.

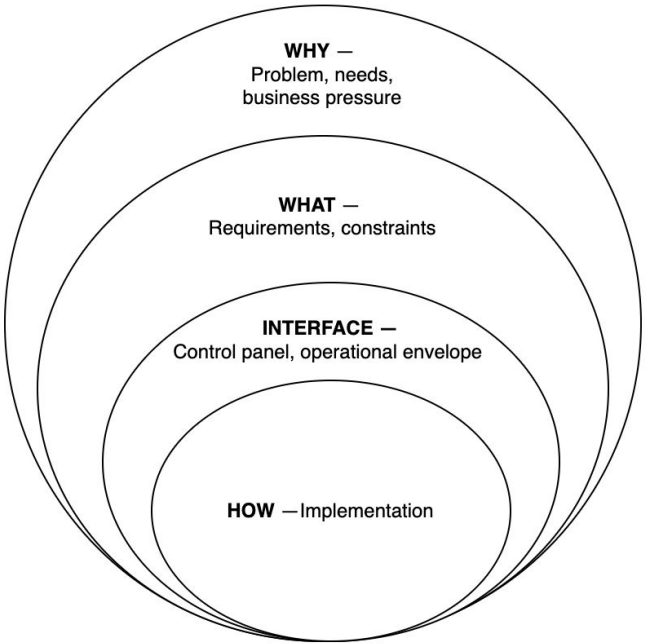
- WHAT and WHY can be especially undervalued and lost in the engineering field.

Let's examine these aspects more closely and see if we can separate them.

# WWH(I) as Onion (WHY — Outer layer, HOW — Core)

WWH consistency principle: A system is consistent if all layers are represented and the direction of influence is preserved:

- WHY defines the intent and drives all other layers.
- WHAT specifies what must be achieved by HOW and is justified by WHY.
- INTERFACE exposes the system in a way that reflects the HOW and keeps it usable.
- HOW implements the system strictly according to WHAT and WHY, and nothing else.



# Quiz: Which aspect is covered by Linux man pages?

| | |
|---|---|
| A. WHAT | B. WHY |
| C. INTERFACE | D. HOW |

```
man sem_wait
```

# Which aspect is covered by Linux man pages?

| | |
|---|---|
| A. WHAT | B. WHY |
| **C. INTERFACE** | D. HOW |

```
man sem_wait
```

## NAME

sem_wait, sem_timedwait, sem_trywait - lock a semaphore

## LIBRARY

POSIX threads library (libpthread, -lpthread)

## SYNOPSIS

```
#include <semaphore.h>

int sem_wait(sem_t *sem);
int sem_trywait(sem_t *sem);
int sem_timedwait(sem_t *restrict sem,
                  const struct timespec *restrict abs_timeout);
```

Feature Test Macro Requirements for glibc (see **feature_test_macros**(7)):

**sem_timedwait**():
_POSIX_C_SOURCE >= 200112L

## DESCRIPTION

**sem_wait**() decrements (locks) the semaphore pointed to by sem. If the semaphore's value is greater than zero, then the decrement proceeds, and the function returns, immediately. If the semaphore currently has the value zero, then the call blocks until either it becomes possible to perform the decrement (i.e., the semaphore value rises above zero), or a signal handler interrupts the call.

**sem_trywait**() is the same as **sem_wait**(), except that if the decrement cannot be immediately performed, then call returns an error (errno set to **EAGAIN**) instead of blocking.

**sem_timedwait**() is the same as **sem_wait**(), except that abs_timeout specifies a limit on the amount of time that the call should block if the decrement cannot be immediately performed. The abs_timeout argument points to a **timespec**(3) structure that specifies an absolute timeout in seconds and nanoseconds since the Epoch, 1970-01-01 00:00:00 +0000 (UTC).

If the timeout has already expired by the time of the call, and the semaphore could not be locked immediately, then **sem_timedwait**() fails with a timeout error (errno set to **ETIMEDOUT**).

If the operation can be performed immediately, then **sem_timedwait**() never fails with a timeout error, regardless of the value of abs_timeout. Furthermore, the validity of abs_timeout is not checked in this case.

## RETURN VALUE

All of these functions return 0 on success; on error, the value of the semaphore is left unchanged, -1 is returned, and errno is

# Requirement statement/rationale examples

**STATEMENT example — WHAT**

*The `sem_wait()` function **shall lock the semaphore** referenced by sem by performing a semaphore lock operation on that semaphore. If the semaphore value is currently zero, then...*

https://pubs.opengroup.org/onlinepubs/9799919799/functions/sem_wait.html

**RATIONALE example — WHY**

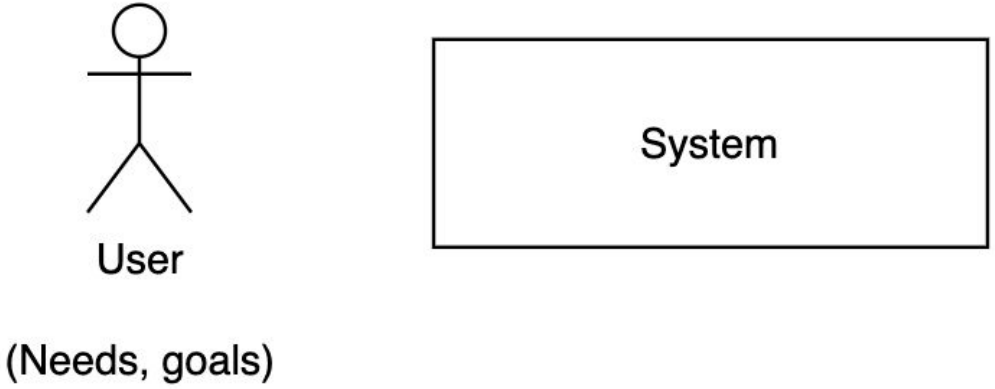
*The `nanosleep()` function specifies that the system-wide clock `CLOCK_REALTIME` is used to measure the elapsed time for this time service. However, with the introduction of the monotonic clock `CLOCK_MONOTONIC` **a new relative sleep function is needed** to allow an application to take advantage of the special characteristics of this clock.*

https://pubs.opengroup.org/onlinepubs/9799919799/functions/clock_nanosleep.html

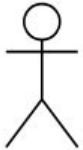
Ideally, requirements should also have a **`UUID`** for easier linking to other elements.

# WHY

# The user has needs/goals (WHY) that the system can address



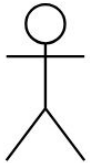
# Problem (WHY) exists independently of any solution



**Problem space:**

- Problem of Environment
- User needs
- Motivation
- Problem statement
- Business goals
- Describes a need, undesired state, "Pain", pressure from reality.

# WHY — Opportunities for solutions

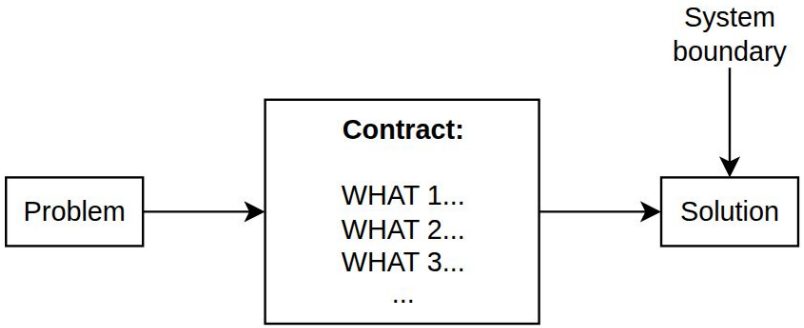


**Problem space:**

- Describes problem without hinting at solution
- Oppotunities for solutions
- Can have multiple solutions, has no obligations
- Only intention to create solution, without requirements
- Describes a problem/situation without system
- No system, no system boundary

# WHAT

# WHAT — Contract for system's behavior a.k.a Requirements

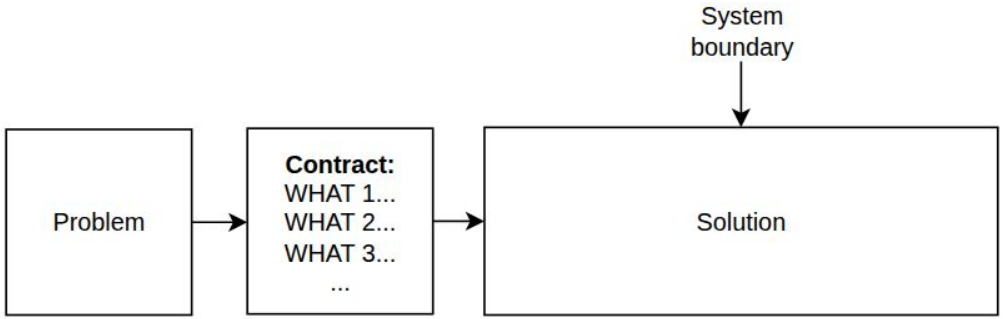



**Contract space:**

- Formal requirements
- "Shall" statements
- Measurable obligations
- Criteria that the solution must meet
- Introduces system boundary
- Reduces solution space
- Multiple solutions possible

# HOW

# HOW — Implementation details

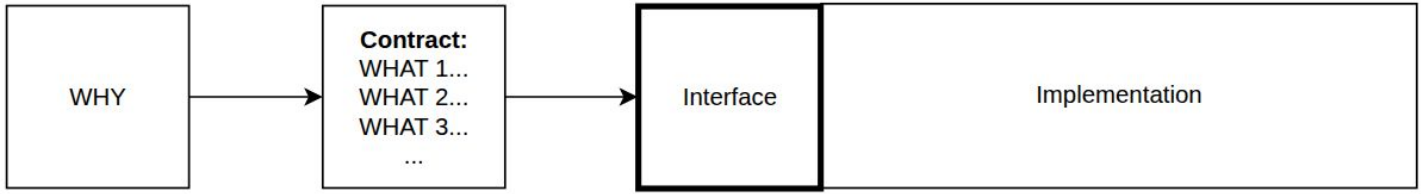



**Solution space:**

- Concrete realization
- Implementation details
- Irreversible technical solution
- Specific design choices
- Internal mechanism and organization
- All decisions are made — The solution is specific

# INTERFACE

# HOW-Interface vs HOW-Implementation




**Solution space (Interface):**

- Point of contact with a system, surface of interaction
- Operational envelope: Allowed inputs, conditions, limits, constraints, assumptions of use
- Parts of the WHAT contract implemented and exposed as outside interface
- No "shall" statements. Instead: interface descriptions
- Examples: GUI, API, Control panel (buttons, display)

# Heuristics for not mixing WHAT-WHY-HOW

# WHY: Solving Right problems

> *"Engineers are great at solving problems but they are not always great at identifying the right problems to be solved."*
> *Dr. John Thomas, ESWC 2019*

# WHY vs WHAT

- **WHY — Needs, WHAT — Formal requirements**
- Boundary and transition between: "Description of problem" → "System obligations"
- Problem vs contract

**Example of mixed WHAT/WHY:**

❌ The system shall validate requirements quickly so engineers can iterate faster.

**Fix:**

✅ WHY: Engineers need fast feedback during editing.
✅ WHAT: The system shall validate modified requirements within 1 second.

# WHY is about Problems, not Behavior

Needs should describe **problems**, not behavior.

Examples:

❌ **Malformed WHY:**
The system needs incremental validation.

✅ **Good WHY:**
Engineers need fast feedback when editing requirements.

# Shall belong to WHAT and nowhere else

- Shall statements belong to the WHAT domain only and nowhere else.
  - **Example:** *The system shall validate modified requirements within 1 second during interactive editing.*
- WHYs are Needs:
  - **Example:** *Reviewers need to understand the intent of changes quickly.*
- I/F descriptions describe the interface surface.
  - **Example:** *The validation service exposes a POST /validate REST endpoint that accepts modified requirements and returns validation diagnostics in JSON format.*
- HOW: Architecture, design, implementation details.
  - **Example:** The component stores requirements in a doubly linked list to support constant-time insertion and removal during incremental validation.

# WHAT vs HOW

- Rule of thumb: WHAT — Requirements, HOW — Design
- Boundary and transition between:
  - "System obligations" ⟶ Concrete realization
- With WHAT:
  - Multiple solutions possible
  - Can be implemented by multiple teams
- With HOW:
  - Irreversible specific solution appears

**Test:**

Could multiple designs satisfy this statement? If no, it probably contains HOW.

## Quiz: Which aspect is covered by Interface Requirements Document (IRD)?

| | |
|---|---|
| A. WHAT | B. WHY |
| C. INTERFACE | D. HOW |

## Quiz: Which aspect is covered by Interface Requirements Document (IRD)?

| | |
|---|---|
| **A. WHAT** | B. WHY |
| C. INTERFACE | D. HOW |

# WHAT vs HOW/Interface

**WHAT (Interface requirement):**

The control panel **shall provide a Play control** that allows the operator to start playback.

**HOW (Interface description)**:

1) The control panel provides a round Play button located in the playback control section of the user interface.
2) The Play button is colored green and labeled with the standard ▶ symbol to indicate playback start.

# HOW: Interface vs Implementation

- Boundary and transition between:
  - "External surface of solution" → "Internal implementation of solution"
- Implementation:
  - If possible to change implementation without breaking outside surface
- Interface:
  - If the change breaks external contract

# WHAT/WHY in HOW

- Relative to Requirements/WHAT, Design documentation is HOW
- But:
  - Design Definition File → The WHAT of HOW
  - Design Justification File → The WHY of HOW
- Architecture Design Records (ADRs):
  - Decision is WHAT
  - Justification is WHY
  - Performed trade-offs is HOW

# WHAT-WHY-HOW in documentation

# WWH(I) applied to industry documents

| WHY | WHAT |
|---|---|
| Business Case<br>Product Brief<br>Product Vision<br>Stakeholder needs<br>Statement of Work<br>Concept of Operations (ConOps)<br>(Top-level) Mission requirements | Project Requirements<br>Functional Specification<br>Technical Requirements Specification<br>Software Requirements Specification<br>High-Level Requirements (HLR)<br>Low-Level Requirements (LLR)<br>Interface Requirements Document (IRD) |
| **INTERFACE** | **HOW** |
| Interface Control Document (ICD)<br>API reference documentation<br>Operator Manual<br>User Manual<br>Safety Manual<br>Instructions manual<br>Flight Operations Manual | Design Definition File (DDF)<br>Design Justification File (DJF)<br>Software Design Description (SDD)<br>Software Design Document (SDD)<br>Architecture Design Document (ADD)<br>Architecture Decision Records (ADR) |

# WHAT-WHY-HOW summary so far

# WWH(I) summary

**WHY — NEEDS**

Problem space — Needs, motivation, problem statement

**Problem of environment:** user needs, business goals, describes a need, undesired state, "Pain", pressure from reality.

**Key characteristics:**

Exists independently from any solution
Oppotunities for solutions
Only intention to create solution, without requirements
Describes problem without hinting at solution

Can have multiple solutions, has no obligations

Describes a problem/situation without system
No system, no system boundary

**WHAT — CONTRACT**

(Scoping) Solution space — Formal response to WHY

**Contract for system behavior:** Formal requirements
Measurable obligations and criteria that the solution must meet.

**Key characteristics:**

Reduces solution space somewhat
Multiple solutions possible

"Shall" statements

Introduces system boundary
Interface requirements are WHAT
Interface descriptions are INTERFACE

**HOW**

**INTERFACE — SYSTEM SURFACE**

Solution space — Point of contact with a system, surface of interaction

**Examples**: UI, API, control panel (buttons, display)

**Operational envelope**: Allowed inputs, conditions, limits, constraints, assumptions of use

**Key characteristics:**

Parts of the WHAT contract implemented and exposed as outside interface
No "shall" statements. Instead: interface descriptions.

**IMPLEMENTATION**

Solution space — Concrete realization

Implementation details
Irreversible technical solution
Specific design choices
Internal mechanism and organization

**Key characteristics:**

All decisions are made — The solution is specific

# Recursive WHY-WHAT-HOW

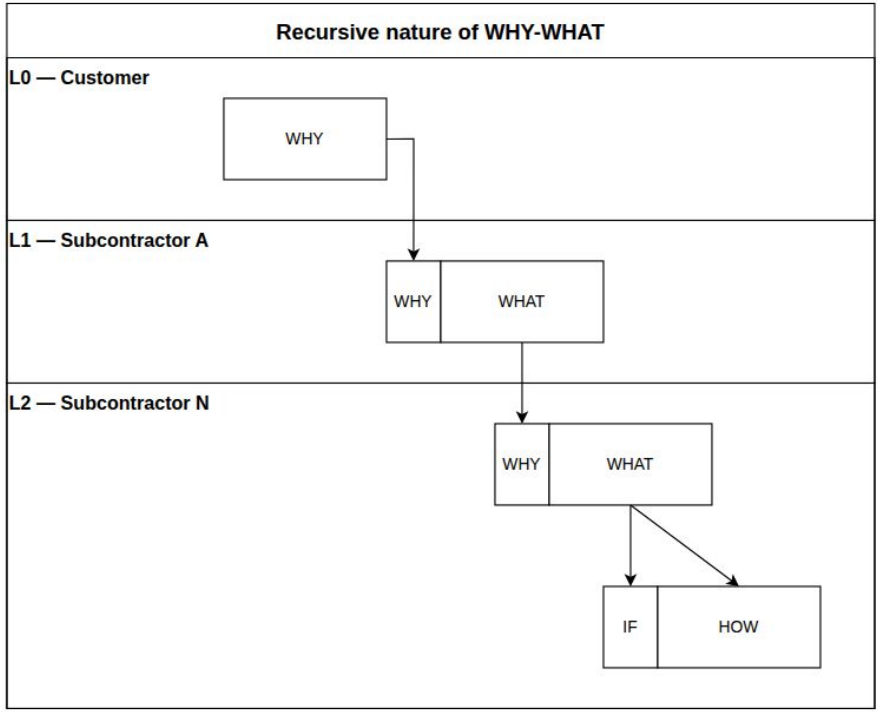
# Recursive WHAT-WHY-HOW

- Every level delegates its HOW to next levels
- Each new level is a HOW for a previous level
- The last level is the actual HOW-implementor

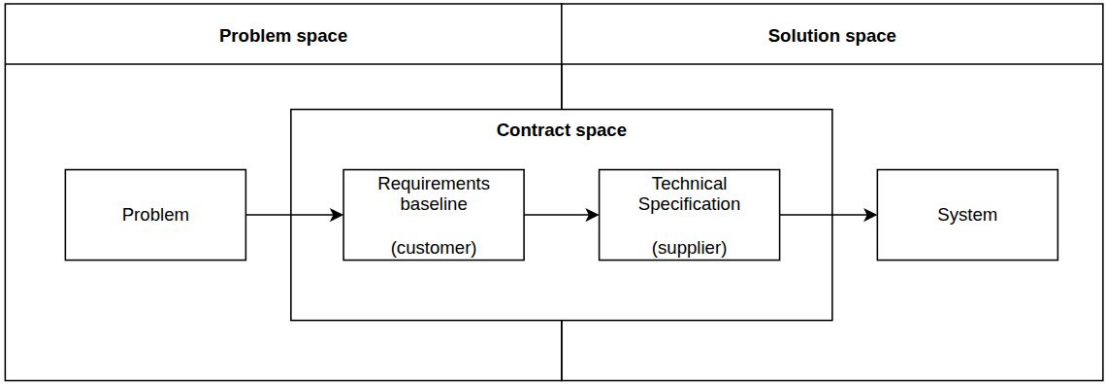
Typical issue:

Missing delegation of HOW to the next level:

→ WHAT absorbs implementation details
→ Loss of clarity and traceability to WHY



# Contract space — European space standard for SW (ECSS-E-ST-40)



# Customer baseline → Supplier specification (ECSS)

**Unclear responsibilities assignment between system and software teams may result into software failures. In particular the interpretation of the "system needs" by the software teams is the cornerstone of the system and software processes.**

A key principle of the ECSS-E-ST-40C to achieve as far as possible the common understanding of the software requirements, is to build the requirements in two steps (RB for needs then TS for solution) and to perform a cross-check through **two separate requirements documents**, i.e. Requirements Baseline (RB) and Technical Specification (TS), and reviews:

1) System Requirements Review to check the needs completeness and

2) the Preliminary Design Review (potentially anticipated by the Software Requirements Review) to allow:

2a) the Customer checking that the proposed software solution answers the needs and

2b) the software supplier checking that his understanding of the customer needs are correct, complete and results into a feasible software solution.

This handbook also recommends an early integration of the software engineering process in the system engineering phase (co engineering) to facilitate the common understanding of the requirements: the supplier can early understand the software needs and checks their feasibility, before the SRR.

ECSS-E-HB-40A, 11 December 2013

# Recursive WHAT-WHY-HOW in requirements

# Example: WHAT-WHY in requirement

| Avionics thermal control | |
|---|---|
| **UID:** | REQ-345 |
| **Links:** | REQ-123, REQ-234 |
| **Statement:** | The spacecraft avionics module shall maintain an operating temperature between −10 °C and +50 °C during all nominal mission phases. |
| **Rationale:** | Electronic components used in the avionics module are qualified to operate within this temperature range; exceeding these limits may cause degraded performance or permanent component failure. |

WHAT

WHY

# Requirement flowdown: (WHAT-WHY) → (WHAT-WHY) → ...

Mission requirements

↓ (system definition)

System requirements (satellite)

↓ (subsystem decomposition)

Subsystem requirements (Satellite subsystems, e.g., Data handling, Power, Thermal, AOCS, Payload)

↓ (software functional decomposition)

High-level software requirements (SW logical functions, e.g., Mission management, File management, etc.)

↓ (software architecture and design)

Low-level software requirements (SW components, e.g., Mission Manager App, File Manager App)

# High-level vs low-level requirements

*DO-178B [27] specifies that the software requirements should be organized into high-level and low-level software requirements. Also, the high-level software requirements should comply with the system requirements (objective 1 of table A-3 in appendix A), and the low-level software requirements should comply with the high-level software requirements (objective 1 of table A-4 in appendix A). These are defined in DO-178B as:*

- *"High-level requirements: Software requirements developed from analysis of system requirements, safety-related requirements, and system architecture.*
- *Low-level requirements: Software requirements derived from high-level requirements, derived requirements, and design constraints from which source code can be directly implemented without further information."*

DOT/FAA/AR-08/32 Requirements Engineering Management Handbook

# Easy heuristic for what low-level software requirements are

*"...Low-level requirements: Software requirements derived from high-level requirements, derived requirements, and design constraints from which source code can be directly implemented without further information."*

**An easy heuristic:** low-level requirements are everything that AI would need to implement a solution without asking any further information.

# Software development with WHAT-WHY-HOW

# Quiz: Which aspect is covered by Doxygen?

| | |
|---|---|
| A. WHAT | B. WHY |
| C. INTERFACE | D. HOW |

# Which aspect is usually covered by Doxygen?

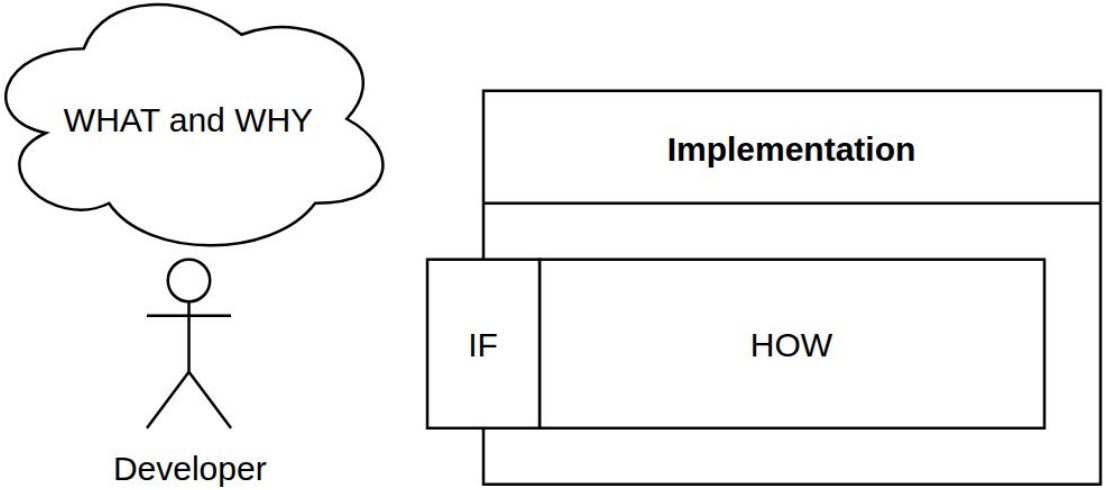
| | |
|---|---|
| A. WHAT | B. WHY |
| **C. INTERFACE** | D. HOW |

The Doxygen conventions are designed to cover INTERFACE. The HOW aspect is sometimes covered as well. The WHAT/WHY are rarely covered.

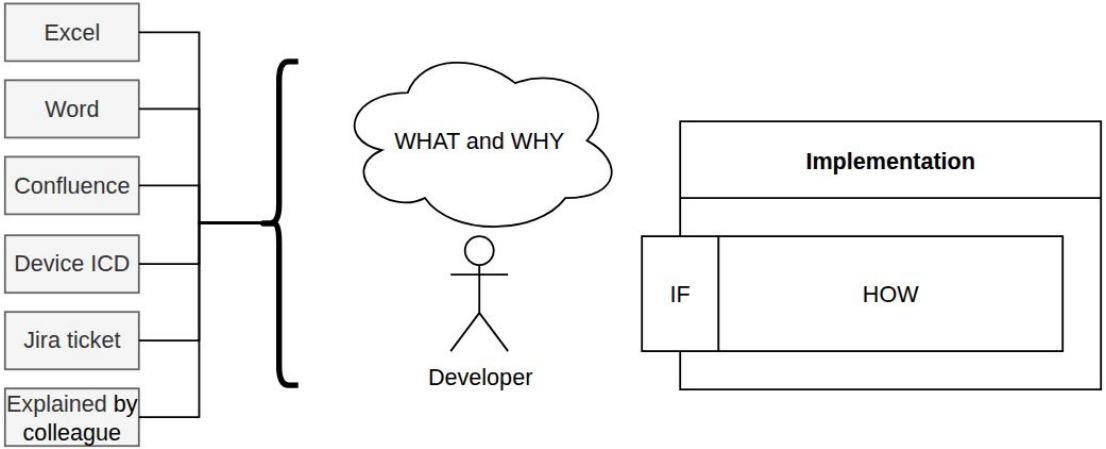
# Doxygen example — Interface

```
/**
 * @brief Calculates the sum of two integers.
 *
 * Adds two integer values and returns the result.
 *
 * @param a First integer value.
 * @param b Second integer value.
 *
 * @return The sum of a and b.
 */
int add(int a, int b);
```

# Naive development with WHAT-WHY-HOW



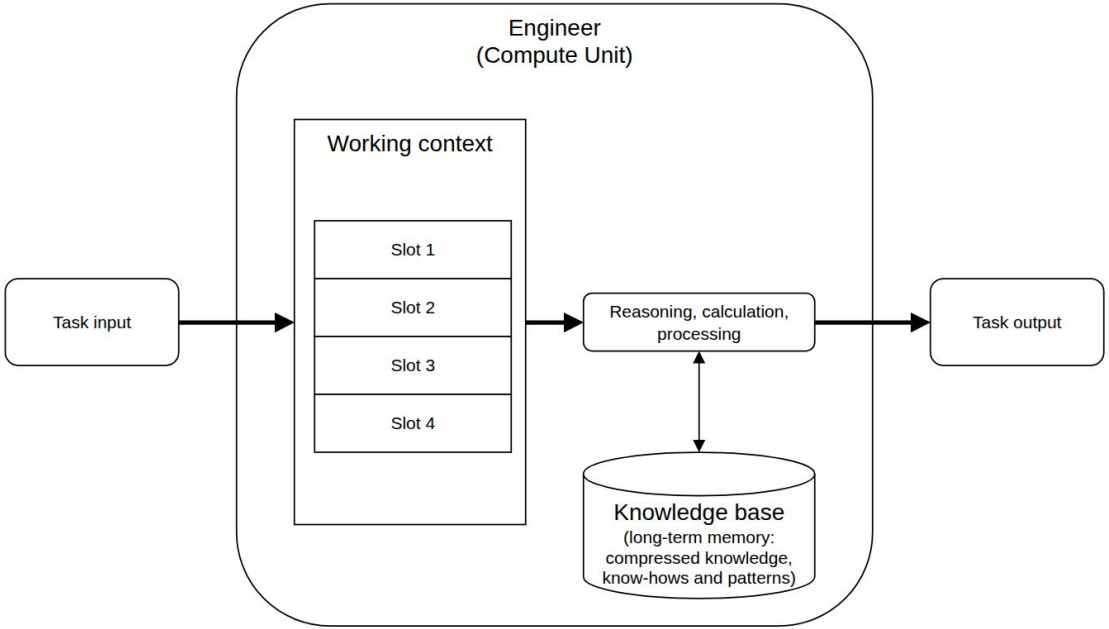
# Naive development with WHAT-WHY-HOW



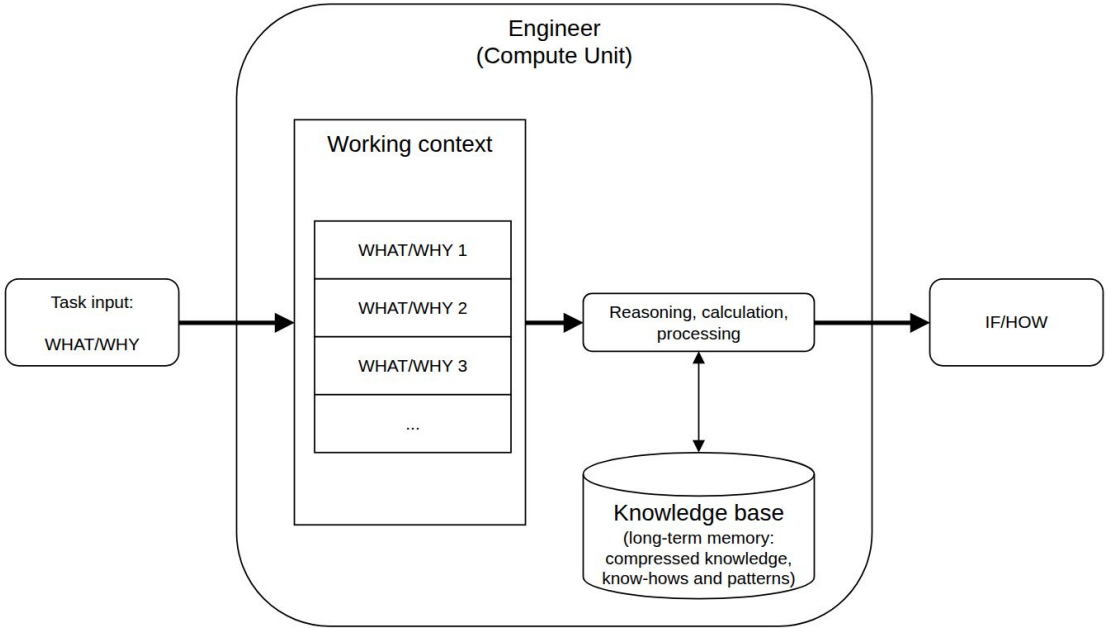
# WWH — Easy? Or not really?

- It is not obvious to everyone, and not everyone agrees to invest effort to fully represent Requirements/WHAT within the realm of formal knowledge.
- Some may think it is self-evident, or not valuable, or that we can easily recall it at any time.

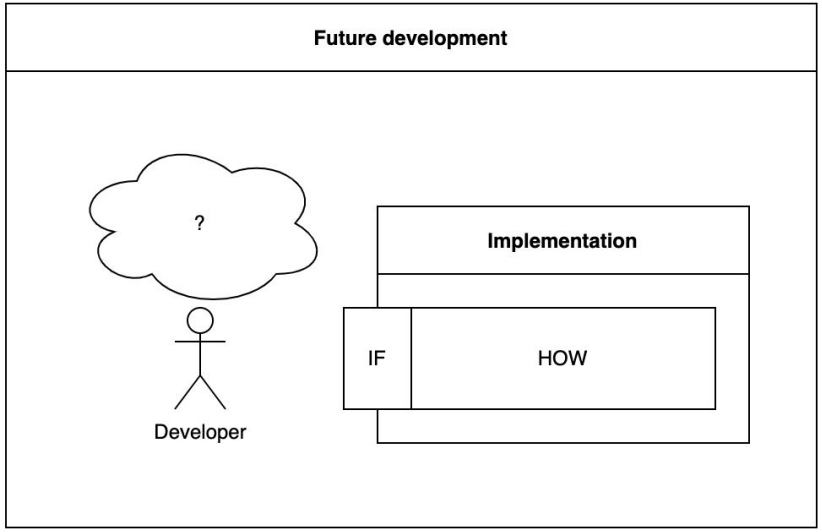
# Engineer as a Compute Unit

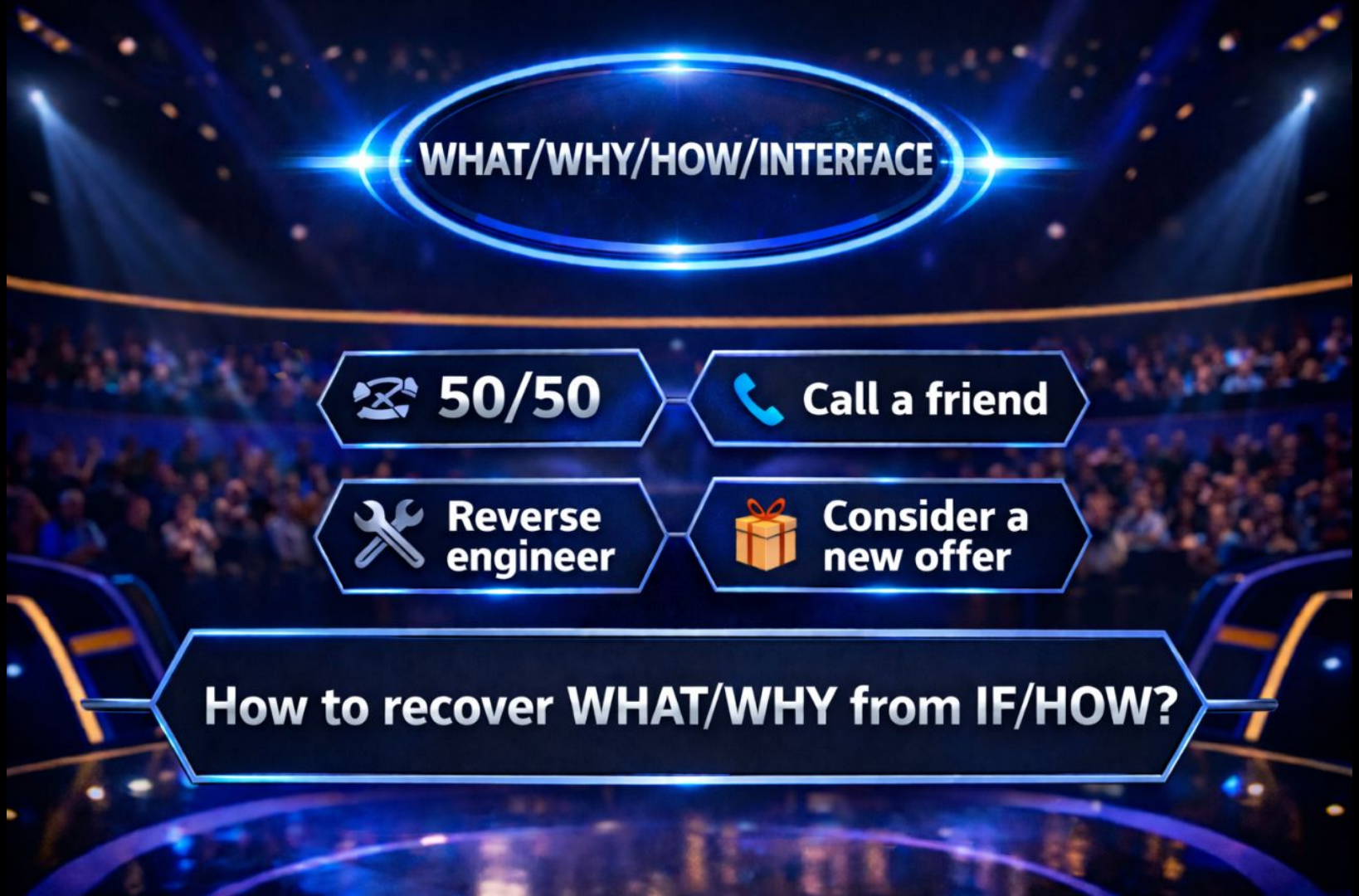


# Engineer as a processor of WHAT/WHY into IF/HOW



# WWH applied to reverse engineering (no WHAT/WHY)



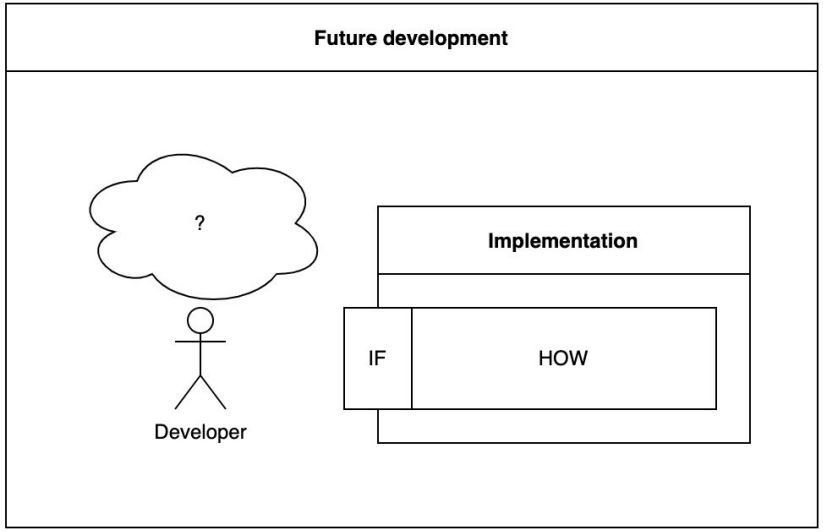

WHAT/WHY/HOW/INTERFACE
50/50
Call a friend
Reverse engineer
Consider a new offer
How to recover WHAT/WHY from IF/HOW?

# No WHAT/WHY – A bold response

| | |
|---|---|
| 50/50 | Call a friend |
| Reverse engineer | Consider a new offer |

*"I don't need any WHAT/WHY, the source of truth is code anyway." —*

Not uncommon for a software engineer.



# Applications of WHAT-WHY-HOW

# Applications — Git commits

```
fix validator
```

VS

```
feat(validator): add incremental validation for changed requirements

WHAT: validate only changed requirements
WHY: full validation is slow on large projects
HOW: detect changed nodes and revalidate dependencies
```

# Applications — Pull/merge requests

**MR: fixes + improvements**

Refactored the validator and cleaned up some stuff.

Also improved performance a bit and fixed a couple of issues related to validation that sometimes happened when editing requirements.

Other small changes:

- refactoring
- improved error messages
- added tests
- some performance improvements
- minor fixes

# Applications — Pull/merge requests

**MR: Incremental validator**

**WHAT**

Validator now runs incrementally instead of validating the entire requirement graph.

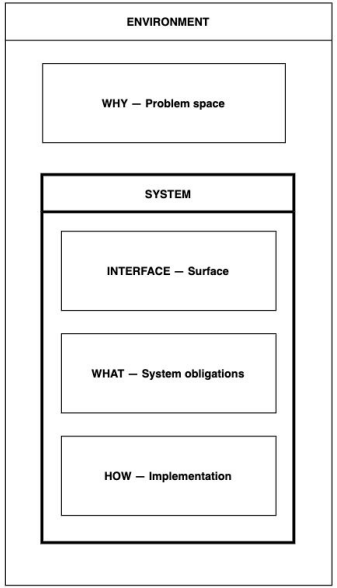
**WHY**

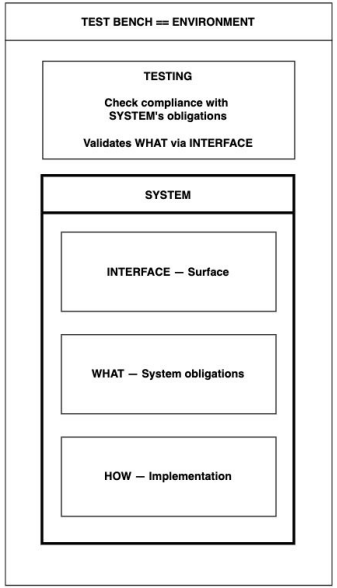
Full validation is slow (~20s on bigger projects) and makes editing painful.
Most edits touch only a few requirements.

**HOW**

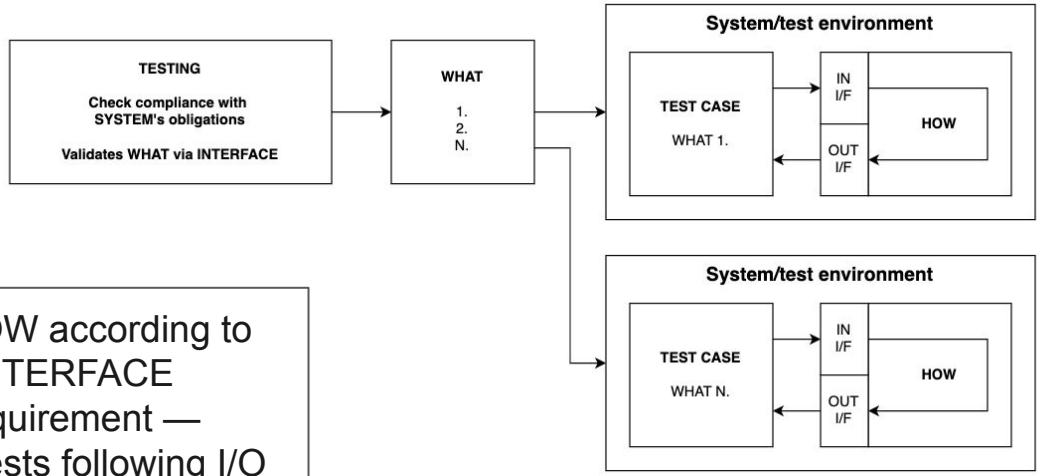
- Track changed requirements
- Revalidate affected nodes
- Split validator into smaller passes
- Improve diagnostics
- Add tests

# WWH(I) applied to testing: Environment → Test bench



# WWH(I) applied to testing: System under test




- Test validates HOW according to WHAT through INTERFACE
- Good testable requirement — Natural to write tests following I/O model

# Key takeaways

- Learn to recognize the WHAT-WHY-HOW aspects in the engineering work
- Recognize when one or more aspects is missing or mixed with other aspects
- The WHAT/WHY are often neglected.
  - HOW only is unlikely to adequately replace and represent the WHAT and WHY
  - Ideally, every work item should come along with WHY it is there
  - Ideally, every work item should come along with traceable WHAT-shall statements
- Without a formal WHAT aspect →
  - Any rigorous or critical software development is weaker, if not impossible
- WWH can be applied:
  - Systems engineering, requirements, architecture
  - Source code and source code comments
  - Commits and pull/merge requests
  - Documentation
  - Technical meetings and discussions